

Fonksiyon

Fonksiyon Nedir?

Bir programın içinde belirli bir işi yapan, adı olan küçük program parçalarıdır.

Programı daha düzenli, okunabilir ve tekrar kullanılabilir hale getirir.

Fonksiyon

- Gerçek dünya problemlerini çözen çoğu bilgisayar programı büyüktür.
- Büyük bir programı geliştirmenin ve sürdürmenin en iyi yolu, onu modül adı verilen küçük ve basit parçalardan oluşturmaktır.
- Bu teknik "böl ve yönet" (divide and conquer) olarak bilinir.
- Böl ve Yönet: Bir programı daha küçük parçalardan veya bileşenlerden oluşturan ve modül denen küçük parçalara bölmek. Her parça, daha kolay yönetilebilir durumdadır.

Fonksiyon

- Her fonksiyon tek bir ve iyi tanımlanmış görevle sınırlandırılmalı ve fonksiyon ismi bu görevi etkili bir şekilde ifade etmelidir.
- Eğer bir fonksiyonu ifade etmek için kısa ve öz bir isim seçilemiyorsa, muhtemelen o fonksiyon çok fazla farklı görevi yerine getirmeye çalışıyordur. Bu tür bir fonksiyonu birkaç küçük fonksiyona bölmek genellikle en iyisidir.

Fonksiyon

1. Parametresiz, değer döndürmeyen

```
#include <stdio.h>
```

```
void hosgeldiniz() {  
    printf("Algoritma ve Programlama dersine hosgeldiniz!\n");  
}
```

```
int main() {  
    hosgeldiniz(); // çağırılıyor  
    printf("Program basladi.\n");  
    return 0;  
}
```

void → fonksiyonun değer döndürmediğini gösterir.
Fonksiyon main içinde çağrılınca ekrana mesaj basar.

Fonksiyon

2. Parametrelili değer döndürmeyen

```
#include <stdio.h>

void kareYaz(int sayi) {
    printf("%d sayisinin karesi: %d\n", sayi, sayi * sayi);
}

int main() {
    int x = 5;
    kareYaz(x); // parametre gönderiliyor
    return 0;
}
```

Fonksiyon

3. Parametresiz – Değer Döndüren

```
#include <stdio.h>
```

```
int sabitDeger() {  
    return 10;  
}
```

```
int main() {  
    int sonuc = sabitDeger();  
    printf("Sonuc: %d\n", sonuc);  
    return 0;  
}
```

-return 10; → fonksiyon sabit bir değeri geri gönderiyor.
-main() bu değeri alıp sonuc değişkenine atıyor.

Fonksiyon

4. Parametrelili – Değer Döndüren

```
#include <stdio.h>

int kare(int x) {
    return x * x;
}

int main() {
    int n;
    printf("Sayi Giriniz:");
    scanf("%d", &n);
    printf("Girilen Sayinin Karesi: %d\n", kare(n));
    return 0;
}
```

kare() fonksiyonu dışarıdan aldığı x'in karesini hesaplayıp döndürür.
main() bunu ekrana bastırır.

Soru: Aşağıdaki kriterlere uygun bir C programı yazınız:

- kare_al adında bir fonksiyon oluşturun: Bu fonksiyon, dışarıdan parametre olarak bir tam sayı (int) almalı, bu sayının karesini hesaplamalı ve sonucu geri döndürmelidir.
- main fonksiyonu içerisinde:
 - 0 ile 9 arasındaki (0 ve 9 dahil) tüm tam sayıları kapsayan bir for döngüsü kurun.
 - Döngünün her adımında, mevcut sayıyı kare_al fonksiyonuna göndererek karesini hesaplatın.
 - Sonuçları ekrana her satıra bir sayı gelecek şekilde; "X'nin karesi = Y" formatında yazdırın.
- Programın sonunda: Ekrana "Program başarıyla tamamlandı." mesajını yazdırarak programı sonlandırın.


```

#include <stdio.h>

/*
    Bu fonksiyon, kendisine verilen tam sayının karesini hesaplar ve döndürür.
    Parametre: y > karesi alınacak sayı (değerle çağrılır, kopya alınır)
    Dönüş: y * y sonucu (int tipinde)
*/
int kare_al(int y) {
    int sonuc;           // geçici değişken

    sonuc = y * y;       // kare hesaplanır

    return sonuc;        // hesaplanan kare döndürülür
}

int main() {
    int x;               // döngüde kullanılacak sayaç / sayı

    /* 0'dan 9'a kadar olan sayıların karelerini ekrana yazdıracağız */
    for (x = 0; x <= 9; x++) {
        // Her adımda: sayı + " karesi = " + kare_al(x) sonucunu yaz
        printf("%d'nin karesi = %d\n", x, kare_al(x));
    }

    printf("\nProgram başarıyla tamamlandı.\n");

    return 0;           // program normal şekilde bittiğini işletim sistemine bildirir
}

```

Soru: Kullanıcı tarafından klavyeden girilen üç tam sayı içerisinde en büyük olanı tespit edip ekrana yazdıran bir C programı kodlayınız.

```

#include <stdio.h>

int maksimum(int x, int y, int z);

/*
   Programın başlangıç noktası
*/
int main() {
    int sayi1, sayi2, sayi3;

    printf("Üç tam sayı giriniz: ");
    scanf("%d %d %d", &sayi1, &sayi2, &sayi3);

    printf("En büyük sayı: %d\n",
           maksimum(sayi1, sayi2, sayi3));

    return 0; // Program başarıyla bitti
}

int maksimum(int x, int y, int z) {
    int en_buyuk = x; // Başlangıçta x'i en büyük kabul et

    if (y > en_buyuk) {
        en_buyuk = y; // y daha büyükse güncelle
    }

    if (z > en_buyuk) {
        en_buyuk = z; // z daha büyükse güncelle
    }

    return en_buyuk; // En büyük değeri çağıran yere geri gönder
}

```

Fonksiyon

```
double average(const int a[], int n) {  
    if (n <= 0) return -1.0;      // contract: geçersiz n  
    long long sum = 0;  
    for (int i = 0; i < n; i++) {  
        if (a[i] < 0 || a[i] > 100) return -1.0; // contract: geçersiz not  
        sum += a[i];  
    }  
    return (double)sum / n;  
}
```

Fonksiyon

```
int max_index(const int a[], int n) {  
    if (n <= 0) return -1;  
    int idx = 0;  
    for (int i = 1; i < n; i++)  
        if (a[i] > a[idx]) idx = i;  
    return idx;  
}
```

Fonksiyon

```
int mode_value(const int a[], int n) {  
    int c[101] = {0};  
    for (int i = 0; i < n; i++) c[a[i]]++;  
  
    int mode = 0;  
    for (int v = 1; v <= 100; v++)  
        if (c[v] > c[mode]) mode = v;  
    return mode;  
}
```

Fonksiyon

```
printf("%-12s | %8.2f\n", "Ortalama", avg);  
printf("%-12s | %8d\n", "Max", scores[idx]);
```

Fonksiyon

Değer ile Çağırma ve Referans ile Çağırma

Değer ile çağırma (Call by value):

- Fonksiyona argümanın bir kopyası aktarılır.
- Fonksiyon içindeki değişiklikler orijinal veriyi etkilemez.
- Fonksiyonun argümanı değiştirmesine gerek olmadığı durumlarda kullanılır.
- Kazara yapılabilecek değişiklikleri önler.

Fonksiyon

Değer ile Çağırma ve Referans ile Çağırma

Referans ile çağırma (Call by reference):

- Orijinal argümanın kendisini aktarır.
- Fonksiyon içindeki değişiklikler orijinal veriyi etkiler.
- Yalnızca güvenilen fonksiyonlarla birlikte kullanılır.

Fonksiyon

```
#include <stdio.h>

void degerDegistir(int sayi) {
    sayi = 100; // Sadece kopya değişir
    printf("Fonksiyon icindeki deger: %d\n", sayi);
}

int main() {
    int x = 10;
    degerDegistir(x); // x'in bir kopyası gönderilir
    printf("Fonksiyon sonrasi main icindeki x: %d\n", x); // x hala 10'dur
    return 0;
}
```

Fonksiyon

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("%d\n", rand());
    printf("%d\n", rand());
    printf("%d\n", rand());
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL)); // seed = şu anki zaman

    printf("%d\n", rand());
    printf("%d\n", rand());
    printf("%d\n", rand());
    return 0;
}
```

Fonksiyon

Kural: min..max (dahil) $\rightarrow \text{min} + \text{rand}() \% (\text{max} - \text{min} + 1)$

$\text{rand}() \% N$ her zaman 0 ile $N-1$ arasında değer üretir.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL)); // her çalıştırmada farklı
    int i;
    for (i = 0; i < 10; i++) {
        int r = 20 + rand() % 11;
        printf("%d ", r);
    }
    printf("\n");
    return 0;
}
```

Fonksiyon

C dilinde; sistem saatini tohum (srand) alarak 1 ile 6 arasında rastgele 20 adet "zar sonucu" üreten, bu sonuçları her satırda 5 sayı olacak şekilde ekrana yazdıran programı kodlayınız.

```
#include <stdio.h>    // printf, scanf gibi giriş/çıkış için
#include <stdlib.h>    // rand() ve srand() için
#include <time.h>      // time() ile tohum (seed) ayarlamak için

int main() {
    int i;            // döngü sayacı

    // Rastgele sayı üreticisini başlat (tohum)
    // Her çalıştırmada farklı sayılar gelsin diye zamanı kullanıyoruz
    srand(time(NULL));

    printf("1 ile 6 arasında 20 tane rastgele zar atisi:\n");

    // 20 kez döngü yap
    for (i = 1; i <= 20; i++) {
        // 1 ile 6 arasında rastgele sayı üret
        // rand() % 6 > 0..5 verir, +1 yapınca 1..6 olur
        int zar = (rand() % 6) + 1;

        printf("%d ", zar);    // sayıyı yazdır (boşluk bırak)

        // Her 5 sayıda bir alt satıra geç (daha okunaklı olsun)
        if (i % 5 == 0) {
            printf("\n");
        }
    }

    printf("\nProgram bitti.\n");

    // Bazı eski sistemlerde konsolun hemen kapanmasını önlemek için
    // getch(); getch();    // < modern IDE'lerde genellikle gerekmez

    return 0;    // Program normal şekilde sona erdi
}
```

Diziler

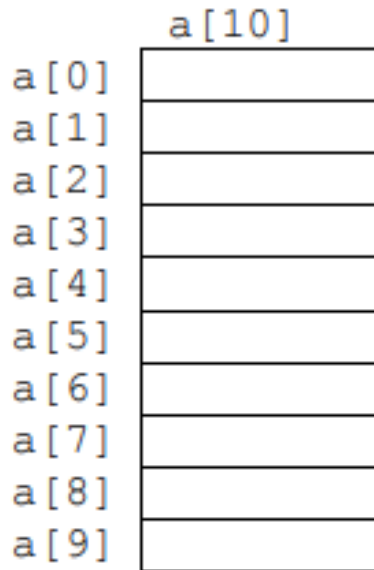
- Aynı türden birden çok değeri tek bir isim altında tutan, sıralı veri yapısıdır.
- Neden var? 50 tane notu not1, not2, ... diye tek tek tutmak yerine not[0..49] ile tutarız: daha kısa, düzenli ve döngülerle işlenebilir.
- İndislemeye 0'dan başlar. İlk eleman a[0], son eleman a[n-1].

Diziler

Tek boyutlu dizilerin bildirimi için kullanılan genel yapı aşağıda gösterilmektedir:

veri-çeşidi ***dizi-adi*** [***boyut***];

`int a[10];`



Diziler

- ✓ Bir diziye başlangıç değeri vermek için, ilgili değişkene o değer doğrudan atanır.
- ✓ **a[0]=2;**
 - dizinin 0 no'lu (ilk) elemanına 2 değerini ata.
- ✓ **a[2]=a[4]+3;**
 - dizinin 2 nolu elemanına 4 nolu elemanın 3 fazlasını ata.
- ✓ **for(i=0;i<6;i++)**
a[i] =0;
 - 0'dan 6'ya kadar dizinin elemanına 0 değerini ata.

Diziler

int b[5] = {10, 20, 30, 40, 50}; // tam doldurma

int c[5] = {1, 2}; // kalanlar 0 olur: {1,2,0,0,0}

int d[] = {3, 4, 5}; // uzunluk 3 otomatik çıkarılır

Diziler

```
#include <stdio.h>

int main()
{
    int n[5], i;
    printf("5 adet sayı giriniz\n");
    for (i=0; i<5; i++)
        scanf("%d", &n[i] );
    printf("Girilen sayılar (sondan başa): \n");
    for (i=4; i>=0; i--)
        printf("%d\n", n[i]);
    return 0;
}
```

5 adet sayı giriniz:

7

16

3

6

5

Girilen sayılar (sondan başa):

5

6

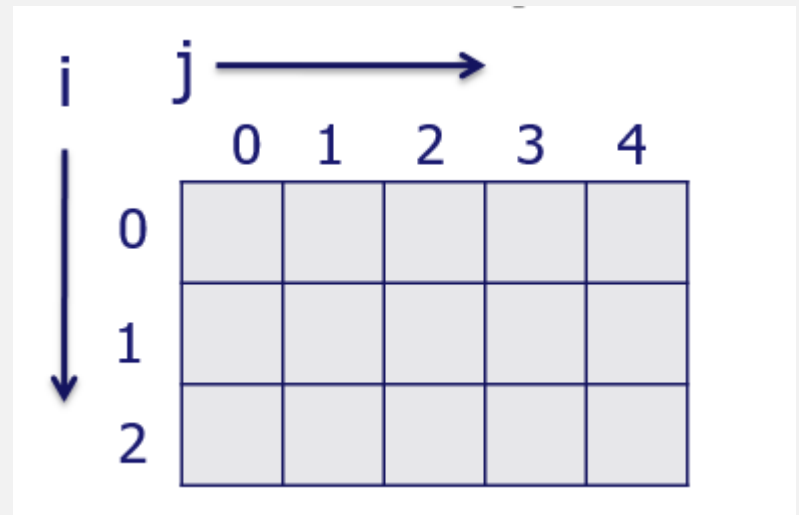
3

16

7

Diziler

- ✓ Bunlardan en sık kullanılanı çift boyutlu dizilerdir.
- ✓ İki boyutlu dizi, satır ve sütunları olan bir tabloya benzer.
- ✓ Bu durumda 1. boyut satırları, 2.boyut ise sütunları gösterecektir.



Diziler

```
int mat[3][4]; // 3 satır, 4 sütun
```

```
int m[2][3] = {  
    {1,2,3},  
    {4,5,6}  
};
```

```
m[0][2] = 99;           // 1. satır 3. sütun
```

Diziler

```
#include <stdio.h>

int f(int x) {
    x = x + 2;
    return x * 2;
}

int main() {
    int a[3] = {1, 2, 3};
    int r = f(a[1]) + f(a[0]);
    printf("%d %d\n", r, a[1]);
    return 0;
}
```

Diziler

```
#include <stdio.h>

int h(int x) {
    if (x % 2 == 0) return x/2;
    return 3*x + 1;
}

int main() {
    int a[4] = {5, 6, 7, 8};

    for (int i = 0; i < 4; i++) {
        a[i] = h(a[i]);
        if (a[i] > 10) a[i] -= (i + 1);
    }

    printf("%d %d %d %d\n", a[0], a[1], a[2], a[3]);
    return 0;
}
```

Diziler

```
#include <stdio.h>

int main() {
    int a[6] = {2,2,1,0,2,1};
    int c[3] = {0};

    for (int i = 0; i < 6; i++) c[a[i]]++;

    printf("%d %d %d\n", c[0], c[1], c[2]);
    return 0;
}
```


Diziler

Soru: C dilinde 10 elemanlı iki farklı tam sayı dizisi kullanarak aşağıdaki işlemleri gerçekleştiren programı yazınız:

- n1 dizisini başlangıçta rastgele değerlerle, n2 dizisini ise tamamen sıfırlarla başlatın.
- Bir döngü yardımıyla n2 dizisinin elemanlarını $2i + 2$ formülüyle doldurun.
- Her iki dizinin içeriğini indis değerleriyle birlikte tablo halinde ekrana yazdırın.
- Son olarak n1 dizisindeki tüm elemanların toplamını hesaplayıp görüntüleyin.

```

#include <stdio.h>
#define SIZE 10      // Dizi boyutu sabiti (10 eleman)
int main() {
    int i;            // döngü sayacı
    int toplam = 0;    // dizideki elemanların toplamı
    // 1. Dizi tanımlama ve başlatma (başlangıç değerleri vererek)
    int n1[SIZE] = {32, 27, 64, 18, 95, 14, 90, 70, 60, 37};
    // 2. Başka bir dizi tanımlama (başlangıç değeri vermeden, sıfırlanır)
    int n2[SIZE] = {0}; // tüm elemanlar otomatik 0 olur
    // 3. n2 dizisini doldurma (0'dan 9'a kadar 2*i + 2 değerleri atanıyor)
    for (i = 0; i < SIZE; i++) {
        n2[i] = 2 * i + 2; // 2, 4, 6, 8, ..., 20
    }
    // 4. n1 dizisinin içeriğini tablo şeklinde yazdırma
    printf("n1 dizisinin elemanlari:\n");
    printf("Indis\tDeger\n");
    printf("-----\t-----\n");
    for (i = 0; i < SIZE; i++) {
        printf("%4d\t%5d\n", i, n1[i]);
    }
    printf("\n");
    // 5. n2 dizisinin içeriğini de yazdırma
    printf("n2 dizisinin elemanlari:\n");
    printf("Indis\tDeger\n");
    printf("-----\t-----\n");
    for (i = 0; i < SIZE; i++) {
        printf("%4d\t%5d\n", i, n2[i]);
    }
    // 6. n1 dizisindeki tüm elemanların toplamını hesaplama
    for (i = 0; i < SIZE; i++) {
        toplam += n1[i];
    }
    printf("\n");
    printf("n1 dizisindeki tum elemanlari toplami = %d\n", toplam);
    return 0;
}

```

Diziler

C dilinde dizilerin ve tekil değişkenlerin fonksiyona gönderilme farkını gösteren bir program yazınız.

- Bir dizinin tamamını fonksiyona gönderip elemanlarını 2 ile çarpan bir fonksiyon yazınız (Oriijinal dizi değişmeli).
- Bir tam sayıyı değer olarak (value) fonksiyona gönderip içerde 2 ile çarpan bir fonksiyon yazınız (Oriijinal değişken değişmemeli).

```

#include <stdio.h>
#define BOYUT 5
// Diziyi parametre olarak alan fonksiyon
// Dizi ismi geçtiği için orijinal dizi üzerinde değişiklik yapılabilir
void diziyiIkiyleCarp(int dizi[], int kacTane) {
    int i;
    for (i = 0; i < kacTane; i++) {
        dizi[i] = dizi[i] * 2;    // < orijinal dizi değişir
    }
}
// Sadece değer kopyalanır, orijinal değişmez
void tekSayiyiIkiyleCarp(int sayi) {
    sayi = sayi * 2;    // < sadece bu fonksiyon içindeki kopya değişir
    printf("Fonksiyon icinde sayi = %d\n", sayi);
}
int main() {
    int sayilar[BOYUT] = {10, 20, 30, 40, 50};
    int j;
    printf("Baslangic hali:\n");
    for (j = 0; j < BOYUT; j++) {
        printf("%d ", sayilar[j]);
    }
    printf("\n\n");
    // 1. Diziyi fonksiyona gönderiyoruz > orijinal değişir
    diziyiIkiyleCarp(sayilar, BOYUT);
    printf("DiziyiIkiyleCarp cagrisindan sonra:\n");
    for (j = 0; j < BOYUT; j++) {
        printf("%d ", sayilar[j]);
    }
    printf("\n\n");
    // 2. Tek bir elemanı (değeri) gönderiyoruz > orijinal değişmez
    int ilkEleman = sayilar[0];    // şu an 20
    printf("Fonksiyona gondermeden once ilkEleman = %d\n", ilkEleman);
    tekSayiyiIkiyleCarp(ilkEleman);
    printf("Fonksiyondan donduktan sonra ilkEleman = %d\n", ilkEleman);
    return 0;
}

```

Diziler

C dilinde, bir sınıftaki öğrencilerin sınav notlarını analiz eden bir program yazınız.

İstenenler

Programınız kullanıcıdan:

1.Önce **öğrenci sayısı N** değerini almalıdır.

- **N 1 ile 100 arasında** olmalıdır.

- Eğer N bu aralığın dışındaysa program:

- ekrana **"Gecersiz N!"** yazdırmalı
- programı sonlandırmalıdır.

2.Ardından **N adet not** girişi alınmalıdır.

- Her not **0–100** aralığında olmalıdır.

- Eğer herhangi bir not bu aralığın dışındaysa program:

- ekrana **"Hata: Notlar 0-100 araliginda olmalı!"** yazdırmalı
- programı sonlandırmalıdır.

Girişler geçerli olduğu durumda program aşağıdaki bilgileri hesaplayıp ekrana yazdırmalıdır:

- Ortalama** (ondalıklı sayı, 2 basamak gösteriniz)

- En yüksek not** ve bu notun **dizideki indeksi**

(Not: indeks 0'dan başlar)

- Geçen öğrenci sayısı** (notu **60 ve üzeri** olanların sayısı)

- Mode (en sık görülen not)** ve bu notun **kaç kez tekrar ettiği**

Diziler

1) Problemin girdilerini ve sınırlarını belirle

- Kullanıcıdan N alınacak.
- N sadece **1–100** aralığında geçerli.
- Dizi boyutu sabit: MAX_N = 100

```
#include <stdio.h>

#define MAX_N 100
```

Diziler

2) Program mimarisini kur: Manager–Worker (main akışı yönetir)

- main **sadece** şu işi yapacak: sırayla fonksiyonları çağırmak + hata kontrolü.
- Hesaplama işlerini fonksiyonlar yapacak.

```
int    input_scores(int a[], int n);
double average(const int a[], int n);
int    max_index(const int a[], int n);
int    count_pass(const int a[], int n, int passMark);
int    mode_value(const int a[], int n);
int    count_value(const int a[], int n, int x);
void    print_report(const int a[], int n, double avg, int maxIdx,
    :      :      :      :      int passCount, int mode, int modeCount);
```

```

int main(void) {
    int n;
    printf("Kac ogrenci notu girilecek? (1-%d): ", MAX_N);
    scanf("%d", &n);

    if (n < 1 || n > MAX_N) {
        printf("Gecersiz N!\n");
        return 1;
    }

    int scores[MAX_N];

    if (!input_scores(scores, n)) {
        printf("Hata: Notlar 0-100 araliginda olmalı!\n");
        return 1;
    }

    double avg = average(scores, n);
    if (avg < 0) {
        printf("Hata: Ortalama hesaplanamadi.\n");
        return 1;
    }

    int maxIdx = max_index(scores, n);
    int passCount = count_pass(scores, n, 60);

    int mode = mode_value(scores, n);
    int modeCount = count_value(scores, n, mode);

    print_report(scores, n, avg, maxIdx, passCount, mode, modeCount);
    return 0;
}

```


Diziler

3) Notları contract'lı oku

- Geçersiz not varsa 0 döndür, main durdursun.

```
int input_scores(int a[], int n) {  
    int i;  
    for (i = 0; i < n; i++) {  
        printf("%d. not (0-100): ", i + 1);  
        scanf("%d", &a[i]);  
        if (a[i] < 0 || a[i] > 100) return 0; // geçersiz not  
    }  
    return 1;  
}
```

Diziler

4) Ortalama (contract + taşma önlemi)

- Hata: -1.0
- Toplam için long long.

```
double average(const int a[], int n) {  
    int i;  
    if (n <= 0) return -1.0;  
  
    long long sum = 0;  
    for (i = 0; i < n; i++) {  
        if (a[i] < 0 || a[i] > 100) return -1.0;  
        sum += a[i];  
    }  
    return (double)sum / n;  
}
```

Diziler

5) max indeks döndür

- değer a[maxIdx] ile alınır

```
int max_index(const int a[], int n) {  
    int i;  
    if (n <= 0) return -1;  
  
    int idx = 0;  
    for (i = 1; i < n; i++)  
        if (a[i] > a[idx]) idx = i;  
  
    return idx;  
}
```

Diziler

6) Geçen sayısını hesapla

- ≥ 60 say (passMark parametre)

```
int count_pass(const int a[], int n, int passMark) {  
    int cnt = 0, i;  
    for (i = 0; i < n; i++)  
        if (a[i] >= passMark) cnt++;  
    return cnt;  
}
```

Diziler

7) Mode'u bul (frekans dizisi)

- mode_value sadece mode değerini döndürür

```
int mode_value(const int a[], int n) {  
    int counts[101] = {0}, i, v;  
  
    for (i = 0; i < n; i++) {  
        if (a[i] >= 0 && a[i] <= 100) counts[a[i]]++;  
    }  
  
    int mode = 0;  
    for (v = 1; v <= 100; v++)  
        if (counts[v] > counts[mode]) mode = v;  
  
    return mode;  
}
```

Diziler

8) Mode sayısını bul: “verilen değeri say”

- Mode’u bulduktan sonra mode’un kaç kez geçtiğini ayrı fonksiyon sayar

```
int count_value(const int a[], int n, int x) {  
    int cnt = 0, i;  
    for (i = 0; i < n; i++)  
        if (a[i] == x) cnt++;  
    return cnt;  
}
```

9) Rapor yazdır

- tablo formati

```
void print_report(const int a[], int n, double avg, int maxIdx,
    int passCount, int mode, int modeCount) {

    printf("\n%-18s | %10s\n", "Metrix", "Deger");
    printf("-----\n");
    printf("%-18s | %10d\n", "N", n);
    printf("%-18s | %10.2f\n", "Ortalama", avg);
    printf("%-18s | %10d\n", "En yuksek", a[maxIdx]);
    printf("%-18s | %10d\n", "Max indeks", maxIdx);
    printf("%-18s | %10d\n", "Gecen sayisi", passCount);
    printf("%-18s | %10d\n", "Mode", mode);
    printf("%-18s | %10d\n", "Mode sayisi", modeCount);
}
```

10) main: sadece akışı yönet (Manager)

- N kontrol
- input kontrol
- pipeline: avg, maxIdx, passCount, mode, modeCount
- rapor


```

int main(void) {
    int n;
    printf("Kac ogrenci notu girilecek? (1-%d): ", MAX_N);
    scanf("%d", &n);

    if (n < 1 || n > MAX_N) {
        printf("Gecersiz N!\n");
        return 1;
    }

    int scores[MAX_N];

    if (!input_scores(scores, n)) {
        printf("Hata: Notlar 0-100 araliginda olmalı!\n");
        return 1;
    }

    double avg = average(scores, n);
    if (avg < 0) {
        printf("Hata: Ortalama hesaplanamadi.\n");
        return 1;
    }

    int maxIdx = max_index(scores, n);
    int passCount = count_pass(scores, n, 60);

    int mode = mode_value(scores, n);
    int modeCount = count_value(scores, n, mode);

    print_report(scores, n, avg, maxIdx, passCount, mode, modeCount);
    return 0;
}

```

Diziler

```
#include <stdio.h>

char a[3][3] = {1,2,3,4,5,6,7,8,9};
int i,j;

main()
{
    for (i=0; i<3; i++)
    {
        for (j=0; j<3; j++)
        {
            printf("  %d", a[i][j] );
        }
        printf("\n");
    }
}
```